Setting Up a Custom TensorFlow Environment with GPU Support

Summary: This tutorial guides you through creating a specialized TensorFlow environment optimized for GPU-accelerated machine learning. You'll learn how to set up a dedicated workspace, create and activate a Python virtual environment, install TensorFlow with CUDA support, and configure it for use in Jupyter notebooks. By following these steps, you'll have an isolated TensorFlow setup ready for deep learning projects, complete with GPU acceleration capabilities and integration with Jupyter.

Edit me

This tutorial guides you through creating a specialized TensorFlow environment optimized for GPU-accelerated machine learning. You'll learn how to set up a dedicated workspace, create and activate a Python virtual environment, install TensorFlow with CUDA support, and configure it for use in Jupyter notebooks. By following these steps, you'll have an isolated TensorFlow setup ready for deep learning projects, complete with GPU acceleration capabilities and integration with Jupyter.

1. Activating the existing Python 3.9 Conda environment:

```
conda activate py39
```

   - `conda activate` is the command to activate a Conda environment
   - `py39` is the name of the pre-existing environment
   - This ensures all subsequent commands use Python 3.9 from this environment

2. Creating a directory:

```
mkdir -p /data/datasets/$USER/
```

   - `mkdir` is the command to make a directory
   - `-p` flag creates parent directories if they don't exist
   - `/data/datasets/$USER/` is the path where `$USER` expands to your username
   - This step ensures you have a personal workspace in a location with ample storage

3. Creating a virtual environment:

```
python -m venv /data/datasets/$USER/venv_tf_2_17
```

   - `python` uses the Python from the activated Conda environment
   - `-m venv` runs the venv module to create a virtual environment
   - This creates an isolated Python environment named `venv_tf_2_17`

4. Activating the virtual environment:

```
source /data/datasets/$USER/venv_tf_2_17/bin/activate
```

   - `source` runs the activate script
   - This activates the virtual environment, ensuring subsequent pip installs are isolated

5. Installing TensorFlow:

```
pip install tensorflow==2.17.0
```

   - Installs TensorFlow version 2.17.0 in the virtual environment

6. Installing CUDA Python:

```
pip install cuda-python==12.3
```

   - Installs CUDA Python, enabling GPU acceleration for certain operations

7. Installing NVIDIA cuDNN:

```
pip install nvidia-cudnn-cu12==8.9.7.29
```

   - Installs NVIDIA's CUDA Deep Neural Network library (cuDNN)
   - This optimizes performance for deep learning on NVIDIA GPUs

8. Installing IPython kernel:

```
pip install ipykernel
```

   - Installs the IPython kernel, which allows using this environment in Jupyter notebooks

9. Setting up the kernel for Jupyter:

```
python -m ipykernel install --user --name tf.2.17
```

   - Installs a new Jupyter kernel named "tf.2.17"
   - `--user` installs it for the current user only
   - This allows you to select this TensorFlow environment in Jupyter notebooks

Site last generated: Oct 22, 2024